

STL

Escaneie para acessar o slide da Aula 3



[luisenilton.github.io/Curso-Maratona/aula3](https://luisenilton.github.io/Curso-Maratona/aula3)

# Sumário da Aula

- |                                   |                                      |
|-----------------------------------|--------------------------------------|
| 1. <u>STL</u>                     | 15. <u>Fila</u>                      |
| 2. <u>Sumário da Aula</u>         | 16. <u>Fila em C++</u>               |
| 3. <u>Ponteiros</u>               | 17. <u>Questão</u>                   |
| 4. <u>Observação</u>              | 18. <u>Algoritmos</u>                |
| 5. <u>Lista ligada</u>            | 19. <u>Sort</u>                      |
| 6. <u>STL</u>                     | 20. <u>Reverse</u>                   |
| 7. <u>Containers</u>              | 21. <u>Swap</u>                      |
| 8. <u>Iteradores</u>              | 22. <u>Unique</u>                    |
| 9. <u>Por que usar Iteradores</u> | 23. <u>Questão</u>                   |
| 10. <u>Pair</u>                   | 24. <u>Referências</u>               |
| 11. <u>Problema</u>               | 25. <u>Contatos</u>                  |
| 12. <u>Pilha</u>                  | 26. <u>Obrigado por acompanhar a</u> |
| 13. <u>Pilha em C++</u>           | <u>aula</u>                          |
| 14. <u>Questão clássica</u>       |                                      |

# Ponteiros

# Ponteiros

Cada variável declarada no programa tem uma localização na memória associada a ela que chamamos de endereço.

# Ponteiros

Cada variável declarada no programa tem uma localização na memória associada a ela que chamamos de endereço.

Ponteiros são variáveis que guardam o endereço de memória de outras variáveis, eles são declarados colocando um '\*' após o tipo.

# Ponteiros

Cada variável declarada no programa tem uma localização na memória associada a ela que chamamos de endereço.

Ponteiros são variáveis que guardam o endereço de memória de outras variáveis, eles são declarados colocando um '\*' após o tipo.

Por exemplo "int\*", indica que esse é um ponteiro que guarda um endereço de uma variável int.

# Ponteiros

Cada variável declarada no programa tem uma localização na memória associada a ela que chamamos de endereço.

Ponteiros são variáveis que guardam o endereço de memória de outras variáveis, eles são declarados colocando um '\*' após o tipo.

Por exemplo "int\*", indica que esse é um ponteiro que guarda um endereço de uma variável int.

Para acessar o endereço de uma variável usamos '&' e para acessar o valor da variável que o ponteiro está apontando usamos '\*'.



# Ponteiros

Cada variável declarada no programa tem uma localização na memória associada a ela que chamamos de endereço.

Ponteiros são variáveis que guardam o endereço de memória de outras variáveis, eles são declarados colocando um '\*' após o tipo.

Por exemplo "int\*", indica que esse é um ponteiro que guarda um endereço de uma variável int.

Para acessar o endereço de uma variável usamos '&' e para acessar o valor da variável que o ponteiro está apontando usamos '\*'.

Não utilizem ponteiros se não for necessário, é bem fácil criar bugs com eles.

# Ponteiros

Cada variável declarada no programa tem uma localização na memória associada a ela que chamamos de endereço.

Ponteiros são variáveis que guardam o endereço de memória de outras variáveis, eles são declarados colocando um '\*' após o tipo.

Por exemplo "int\*", indica que esse é um ponteiro que guarda um endereço de uma variável int.

Para acessar o endereço de uma variável usamos '&' e para acessar o valor da variável que o ponteiro está apontando usamos '\*'.

Não utilizem ponteiros se não for necessário, é bem fácil criar bugs com eles.

```
int x = 2;
cout << &x << endl; // vai printar um hexadecimal (endereço).

int* ptr = &x;

cout << *ptr << endl; // vai printar 2.

*ptr = 3; // tambem podemos alterar o valor da variavel com o ponteiro
```

# Observação

Qual a complexidade para apagar um elemento de um array?

Lista ligada

# Lista ligada

Listas ligadas é uma estrutura de dados sequencial em que cada elemento tem um ponteiro apontando para o próximo elemento da lista.

# Lista ligada

Listas ligadas é uma estrutura de dados sequencial em que cada elemento tem um ponteiro apontando para o próximo elemento da lista.

Ao contrário do array os elementos não estão em posições contiguas de memória, por isso não conseguimos acessar um índice  $i$  arbitrário.

# Lista ligada

Listas ligadas é uma estrutura de dados sequencial em que cada elemento tem um ponteiro apontando para o próximo elemento da lista.

Ao contrário do array os elementos não estão em posições contiguas de memória, por isso não conseguimos acessar um índice  $i$  arbitrário.

Por outro lado é extremamente fácil remover elementos, já que precisamos apenas retirar a referência aquela posição.

# Lista ligada

Listas ligadas é uma estrutura de dados sequencial em que cada elemento tem um ponteiro apontando para o próximo elemento da lista.

Ao contrário do array os elementos não estão em posições contiguas de memória, por isso não conseguimos acessar um índice  $i$  arbitrário.

Por outro lado é extremamente fácil remover elementos, já que precisamos apenas retirar a referência aquela posição.

Visualização

---



STL

# STL

Significa "Standard Template Library" (Biblioteca de modelos padrão).

# STL

Significa "Standard Template Library" (Biblioteca de modelos padrão).

É uma biblioteca em que já estão implementados estruturas de dados, funções e algoritmos que são essenciais para a maratona.

# STL

Significa "Standard Template Library" (Biblioteca de modelos padrão).

É uma biblioteca em que já estão implementados estruturas de dados, funções e algoritmos que são essenciais para a maratona.

Essas funcionalidades são implementadas da forma genérica possível para serem de propósito geral.

# STL

Significa "Standard Template Library" (Biblioteca de modelos padrão).

É uma biblioteca em que já estão implementados estruturas de dados, funções e algoritmos que são essenciais para a maratona.

Essas funcionalidades são implementadas da forma genérica possível para serem de propósito geral.

Para usa-la precisamos apenas incluir o header "`<bits/stdc++.h>`".

# STL

Significa "Standard Template Library" (Biblioteca de modelos padrão).

É uma biblioteca em que já estão implementados estruturas de dados, funções e algoritmos que são essenciais para a maratona.

Essas funcionalidades são implementadas da forma genérica possível para serem de propósito geral.

Para usa-la precisamos apenas incluir o header "<bits/stdc++.h>".

```
#include <bits/stdc++.h>
```

```
using namespace std; // isso é necessário para não ter que escrever std:: toda vez que for usar.
```

# Containers

# Containers

Containers são estruturas que guardam uma coleção de um tipo específico, na STL os containers são criados de forma genérica permitindo que você defina o seu tipo na criação.



# Containers

Containers são estruturas que guardam uma coleção de um tipo específico, na STL os containers são criados de forma genérica permitindo que você defina o seu tipo na criação.

Exemplo:

# Containers

Containers são estruturas que guardam uma coleção de um tipo específico, na STL os containers são criados de forma genérica permitindo que você defina o seu tipo na criação.

Exemplo:

```
vector<int> a; // ao criarmos o vector definimos o seu tipo com <TIP0>
```

```
pair<int,string> // Definimos os dois tipos do pair com <TIP01,TIP02>
```

# Containers

Containers são estruturas que guardam uma coleção de um tipo específico, na STL os containers são criados de forma genérica permitindo que você defina o seu tipo na criação.

Exemplo:

```
vector<int> a; // ao criarmos o vector definimos o seu tipo com <TIP0>

pair<int,string> // Definimos os dois tipos do pair com <TIP01,TIP02>
```

Em C++, em geral temos 3 tipos de containers:

# Containers

Containers são estruturas que guardam uma coleção de um tipo específico, na STL os containers são criados de forma genérica permitindo que você defina o seu tipo na criação.

Exemplo:

```
vector<int> a; // ao criarmos o vector definimos o seu tipo com <TIP0>

pair<int,string> // Definimos os dois tipos do pair com <TIP01,TIP02>
```

Em C++, em geral temos 3 tipos de containers:

- Sequenciais.
- Associativos.
- Associativos não ordenados.

# Iteradores

# Iteradores

Um iterador é um ponteiro que representa a posição de um elemento num container, é usado para iterar numa estrutura.

# Iteradores

Um iterador é um ponteiro que representa a posição de um elemento num container, é usado para iterar numa estrutura.

Existem tipos diferentes de iteradores, mais detalhes em [Iteradores](#)

# Iteradores

Um iterador é um ponteiro que representa a posição de um elemento num container, é usado para iterar numa estrutura.

Existem tipos diferentes de iteradores, mais detalhes em [Iteradores](#)

## Métodos padrões



# Iteradores

Um iterador é um ponteiro que representa a posição de um elemento num container, é usado para iterar numa estrutura.

Existem tipos diferentes de iteradores, mais detalhes em [Iteradores](#)

## Métodos padrões

- `begin()` , ponteiro para a primeira posição do container.
- `end()` , ponteiro para o final do container.
- `++`, anda uma posição para frente.

# Iteradores

Um iterador é um ponteiro que representa a posição de um elemento num container, é usado para iterar numa estrutura.

Existem tipos diferentes de iteradores, mais detalhes em [Iteradores](#)

## Métodos padrões

- `begin()` , ponteiro para a primeira posição do container.
- `end()` , ponteiro para o final do container.
- `++`, anda uma posição para frente.

```
1 vector<int> a;  
2  
3 vector<int>::iterator itr = a.begin();  
4 vector<int>::iterator fin = a.end();  
5  
6 for(;itr != fin;itr++){  
7     // executa
```

# Iteradores

Um iterador é um ponteiro que representa a posição de um elemento num container, é usado para iterar numa estrutura.

Existem tipos diferentes de iteradores, mais detalhes em [Iteradores](#)

## Métodos padrões

- `begin()` , ponteiro para a primeira posição do container.
- `end()` , ponteiro para o final do container.
- `++`, anda uma posição para frente.

```
1 vector<int> a;  
2 // na pratica utilizamos o auto para não ter que escreve muitas linhas de código  
3 auto itr = a.begin();  
4 auto fin = a.end();  
5  
6 for(;itr != fin;itr++){  
7     // executa
```

# Por que usar Iteradores

# Por que usar Iteradores

Alguém pode estar pensando "Ué, mas eu já itero só com um for, porque eu preciso disso?".

# Por que usar Iteradores

Alguém pode estar pensando "Ué, mas eu já itero só com um for, porque eu preciso disso?".

A grande vantagem de iteradores é que permite iterar estrutura que não são sequenciais (Arrays) como árvores sem ter que modificar a forma que escreve o código.

# Por que usar Iteradores

Alguém pode estar pensando "Ué, mas eu já itero só com um for, porque eu preciso disso?".

A grande vantagem de iteradores é que permite iterar estrutura que não são sequenciais (Arrays) como árvores sem ter que modificar a forma que escreve o código.

Outra vantagem é que algoritmos da STL como `std::sort()` recebem iteradores como parâmetro.

Pair



# Pair

Pair é uma estrutura que guarda dois elementos, os tipos desses elementos é definido na criação.

# Pair

Pair é uma estrutura que guarda dois elementos, os tipos desses elementos é definido na criação.

## Sintaxe

# Pair

Pair é uma estrutura que guarda dois elementos, os tipos desses elementos é definido na criação.

## Sintaxe

```
pair<int,string> par;  
  
// atribuição  
par = {2,"Nome"};  
  
cout << par.first << endl; // 2  
cout << par.second << endl; // Nome  
  
par.first = 3; // podemos modificar um elemento separadamente.  
par.second = "Nome2";
```

# Pair

Pair é uma estrutura que guarda dois elementos, os tipos desses elementos é definido na criação.

## Sintaxe

```
pair<int,string> par;  
  
// atribuição  
par = {2,"Nome"};  
  
cout << par.first << endl; // 2  
cout << par.second << endl; // Nome  
  
par.first = 3; // podemos modificar um elemento separadamente.  
par.second = "Nome2";
```

Para comparar pair, primeiro o programa analisa o primeiro elemento, se os dois forem iguais compara o segundo.

# Pair

Pair é uma estrutura que guarda dois elementos, os tipos desses elementos é definido na criação.

## Sintaxe

```
pair<int,string> par;  
  
// atribuição  
par = {2,"Nome"};  
  
cout << par.first << endl; // 2  
cout << par.second << endl; // Nome  
  
par.first = 3; // podemos modificar um elemento separadamente.  
par.second = "Nome2";
```

Para comparar pair, primeiro o programa analisa o primeiro elemento, se os dois forem iguais compara o segundo.

```
pair<int,int> a = {2,5};  
pair<int,int> b = {3,4};  
cout << (a < b) << endl; // true
```

# Problema

Dado um array A não ordenado, imprima a posição original de cada elemento no array ordenado.

[Link Problema](#)

Pilha

# Pilha

Uma pilha é uma estrutura de dados abstrata que segue o princípio LIFO (Last In First Out), ou seja o ultimo elemento adicionado vai ser o primeiro a ser removido.



# Pilha

Uma pilha é uma estrutura de dados abstrata que segue o princípio LIFO (Last In First Out), ou seja o ultimo elemento adicionado vai ser o primeiro a ser removido.

Uma forma de visualizar é pensar numa pilha de pratos uma em cima da outra, e você pode:

# Pilha

Uma pilha é uma estrutura de dados abstrata que segue o princípio LIFO (Last In First Out), ou seja o ultimo elemento adicionado vai ser o primeiro a ser removido.

Uma forma de visualizar é pensar numa pilha de pratos uma em cima da outra, e você pode:

- Colocar um novo prato em cima.
- Retirar um prato.

# Pilha

Uma pilha é uma estrutura de dados abstrata que segue o princípio LIFO (Last In First Out), ou seja o ultimo elemento adicionado vai ser o primeiro a ser removido.

Uma forma de visualizar é pensar numa pilha de pratos uma em cima da outra, e você pode:

- Colocar um novo prato em cima.
- Retirar um prato.

Se você quiser pegar o prato que está na base, você terá que retirar todos os pratos.

# Pilha

Uma pilha é uma estrutura de dados abstrata que segue o princípio LIFO (Last In First Out), ou seja o ultimo elemento adicionado vai ser o primeiro a ser removido.

Uma forma de visualizar é pensar numa pilha de pratos uma em cima da outra, e você pode:

- Colocar um novo prato em cima.
- Retirar um prato.

Se você quiser pegar o prato que está na base, você terá que retirar todos os pratos.

## Aplicações básicas

# Pilha

Uma pilha é uma estrutura de dados abstrata que segue o princípio LIFO (Last In First Out), ou seja o ultimo elemento adicionado vai ser o primeiro a ser removido.

Uma forma de visualizar é pensar numa pilha de pratos uma em cima da outra, e você pode:

- Colocar um novo prato em cima.
- Retirar um prato.

Se você quiser pegar o prato que está na base, você terá que retirar todos os pratos.

## Aplicações básicas

- Inverter uma string (ou array).
- Botão de voltar do navegador.

# Pilha em C++

```
stack<int> pilha; // voce define o tipo com <TIPO>

pilha.push(2); // adiciona um elemento
pilha.push(3);
cout << pilha.top() << endl; // retorna o elemento no topo da pilha
cout << pilha.size() << endl; // retorna o tamanho atual da pilha
cout << pilha.empty() << endl; // retorna um booleano (se a pilha está vazia).
pilha.pop() // remove o elemento do topo.
```

# Questão clássica

Dada uma sequência de parênteses, determine se ela está balanceada. Ela está balanceada se cada parentêse aberto '(' tem um parêntese fechado correspondente ')'.  
  
Exemplos:

Problema

-----

Fila



# Fila

Uma fila é uma estrutura de dados abstrata que segue o princípio FIFO (First In First Out), ou seja o primeiro elemento adicionado vai ser o primeiro a ser removido.

# Fila

Uma fila é uma estrutura de dados abstrata que segue o princípio FIFO (First In First Out), ou seja o primeiro elemento adicionado vai ser o primeiro a ser removido.

É uma abstração muito utilizada para simular algoritmos e como base para algoritmos mais complexos.

# Fila

Uma fila é uma estrutura de dados abstrata que segue o princípio FIFO (First In First Out), ou seja o primeiro elemento adicionado vai ser o primeiro a ser removido.

É uma abstração muito utilizada para simular algoritmos e como base para algoritmos mais complexos.

Também é a base para algoritmos de busca em largura (falaremos sobre na aula de grafos).

# Fila em C++

```
queue<int> fila; // voce define o tipo com <TIPO>

fila.push(2); // adiciona um elemento
fila.push(3);
cout << fila.front() << endl; // retorna o elemento na frente da fila
cout << fila.size() << endl; // retorna o tamanho atual da fila
cout << fila.empty() << endl; // retorna um booleano (se a fila está vazia).
fila.pop() // remove o elemento do frente.
```

# Questão

Questão

# Algoritmos

# Algoritmos

A STL também tem uma coleção de algoritmos úteis para resolução de problemas.

# Algoritmos

A STL também tem uma coleção de algoritmos úteis para resolução de problemas.

Os principais são:



# Algoritmos

A STL também tem uma coleção de algoritmos úteis para resolução de problemas.

Os principais são:

- `sort()`, ordena um conjunto de elementos (vector ou array) em  $O(N\log N)$
- `reverse()`, inverte um conjunto de elementos em  $O(N)$ .
- `swap()`, troca o valor de duas variáveis em  $O(1)$ .
- `unique()`, dado um array ordenado, ele deixa apenas valores únicos nele.

# Algoritmos

A STL também tem uma coleção de algoritmos úteis para resolução de problemas.

Os principais são:

- `sort()`, ordena um conjunto de elementos (vector ou array) em  $O(N\log N)$
- `reverse()`, inverte um conjunto de elementos em  $O(N)$ .
- `swap()`, troca o valor de duas variáveis em  $O(1)$ .
- `unique()`, dado um array ordenado, ele deixa apenas valores únicos nele.

Lista completa

# Sort

Função que ordena um array de elementos, ela recebe um ponteiro pro início e um ponteiro pro final do array e ordena esse intervalo.

```
1  int n; cin >> n;
2  vector<int> a(n);
3  for(int i = 0 ; i < n;i++){
4      cin >> a[i];
5  }
6  sort(a.begin(),a.end());
7  for(int x : a)cout << x << " ";
8  cout << endl; // 1 2 3 4 5
```

# Sort

Função que ordena um array de elementos, ela recebe um ponteiro pro início e um ponteiro pro final do array e ordena esse intervalo.

```
1  int n; cin >> n;
2  int a[n];
3  for(int i = 0 ; i < n;i++){
4      cin >> a[i];
5  }
6  sort(a,a + n);
7  for(int i = 0 ; i < n;i++) cout << a[i] << " ";
8  cout << endl;
```

# Reverse

Inverte o conteúdo de um vector.

```
int n; cin >> n;
vector<int> a(n);
for(int i = 0 ; i < n;i++){
    cin >> a[i];
}
reverse(a.begin(),a.end()); // inverte o vector
```

# Swap

Troca o valor de duas variáveis em  $O(1)$ .

```
int a = 2, b = 3;
swap(a,b);
cout << a << " " << b << endl;

string s1 = "primeira", s2 = "segunda";
swap(s1,s2);

cout << s1 << " " << s2 << endl;

vector<int> v1 = {2,3,4} , v2 = {6,3};
swap(v1,v2); // troca os dois vetores.
```

# Unique

Dado um array ordenado, essa função move os elementos duplicados pro final, utilizamos o erase pra conseguir o array sem duplicatas.

```
vector<int> a = {2, 3 , 1, 4 , 2 , 1};

// antes de usarmos a função temos que ordenar
sort(a.begin(),a.end()); // {1,1,2,2,3,4};

// a função retorna um ponteiro para o final dos elementos únicos
auto last = unique(a.begin(),a.end()); // {1,2,3,4,x,x}.
// agora precisamos apagar os elementos duplicados

a.erase(last,a.end());

for(int x : a){
    cout << x << " ";
}
cout << endl;
```

# Questão

Descobrir quantos valores únicos eu tenho num array.

Problema

---



# Referências

- Documentação C++
- Programiz
- Livro CPH
- Aula UFMG

# Contatos

Entre no discord para dúvidas



Entre no grupo de questões



# Obrigado por acompanhar a aula

Por favor preencha o formulário de feedback com sugestões e críticas para a próxima aula 🙏.

